



Part 1

Database Anomalies and Normalization

Objectives

- Define database anomaly, and its frequency of occurrence.
- Identify different types of data anomalies.
- Discuss how to address different types of anomalies.
- Discuss the fundamental types of Database Normalization.

What is a database anomaly?

- Anomaly means inconsistency in the pattern from the normal form.
- In a database management system (DBMS), anomaly means an inconsistency that occurs in the relational table during the operations performed on the relational table.
- Anomalies in the relational model refer to inconsistencies or errors that can arise when working with relational databases, specifically in data insertion, deletion, and modification.

What are the causes of database anomalies?

- Database anomalies are the faults in the database caused due to poor management of storing everything in the database.
- If a table is constructed poorly, then there is a chance of database anomaly.
- *Due to database anomalies, the integrity of the database suffers.*

What are the causes of database anomalies?

- The other reason for the database anomalies is that all the data is stored in a single table.
- For example, if there is a lot of redundant data in our database, then DBMS anomalies can occur.
- *Redundant data wastes disk space and creates maintenance problems.*
- If data that exists in more than one place must be changed, the data must be changed in exactly the same way in all locations.

Anomalies in Relational Model

3 Types of Anomalies

- Insertion Anomalies
- Deletion Anomalies
- Update Anomalies

Insertion Anomalies

- Insertion anomalies happen when vital data is inserted into the database, which is not possible because other data is not already there.
- For example, if a system is designed to require that a customer be on file before a sale can be made to that customer, but you cannot add a customer until they have bought something, then you have an insert anomaly.

Insertion Anomaly

P.K.



e_id	name	age	dept_id	dept_name	dept_head	dept_phone
1	abc	34	7	Design	pqr	123
2	mno	27	7	Design	pqr	123
?			10	production	ghi	789

(NEW) PRODUCTION DEAPRTMENT → 0 EMPLOYEE'S

- dept_id (E.G. 10)
- dept_name (E.G. Production)
- dept_head (E.G. ghi)
- dept_phone (E.G. 789)

Deletion Anomalies

- This anomaly happens when the deletion of unwanted information causes desired information to be deleted as well.
- For example, if a single database record contains information about a particular product along with information about a salesperson for the company and the salesperson quits, then information about the product is deleted along with the salesperson's information.

Sample Table

e_id	name	age	dept_id	dept_name	dept_head	dept_phone
1	abc	34	7	Design	pqr	123
2	mno	27	7	Design	pqr	123

making IT simple

Deletion Anomaly- Sample Data

Table Name: emp

e_id	name	age	dept_id	dept_name	dept_head	dept_phone
1	abc	34	7	Design	pqr	123
2	mno	27	7	Design	pqr	123
3	xyz	26	10	Production	tuv	987

making IT simple

Deletion Anomaly

MAJOR 3 ANOMALIES (PROBLEMS): 1) INSERTION ANOMALY 2) DELETION ANOMALY 3) UPDATION ANOMALY

Table Name: emp

e_id	name	age	dept_id	dept_name	dept_head	dept_phone
1	abc	34	7	Design	pqr	123
2	mno	27	7	Design	pqr	123

???

making IT simple



COMPANY



~~PRODUCTION (DEPARTMENT)~~



~~XYZ (EMPLOYEE)~~ LEFT THE JOB

Delete Query:

delete from emp where id = 3;

Update Anomalies

- **Update Anomalies** happen when the person charged with keeping all the records current and accurate is asked, for example, to change an employee's title due to a promotion.
- If the data is stored redundantly in the same table, and the person misses any of them, then multiple titles will be associated with the employee.
- The end-user has no way of knowing which is the correct title.

Update Anomaly Sample Data

Table Name: emp

e_id	name	age	dept_id	dept_name	dept_head	dept_phone
1	abc	34	7	Design	pqr	123
2	mno	27	7	Design	pqr	123
3	xyz	26	10	Production	tuv	987
4	slr	40	10	Production	tuv	987
5	smg	35	3	R & D	ksi	555
6	dmr	29	3	R & D	ksi	555

Update Anomaly

Table Name: emp

e_id	name	age	dept_id	dept_name	dept_head	dept_phone
1	abc	34	7	Design	pqr	123
2	mno	27	7	Design	pqr	123
3	xyz	26	10	Production	tuv	987
4	slr	40	10	Production	tuv	987
5	smg	35	3	R & D	1 msd	555
6	dmr	29	3	R & D	2 ksi	555



R & D DEPARTMENT
HEAD:

~~ksi~~
msd



How do we address database anomalies?

- Generally, database anomalies are removed by the process of ***NORMALIZATION*** which is performed by splitting/joining of tables.

What is Database Normalization?

- Normalization is the process of organizing data in a database.
- It includes creating tables and establishing relationships between those tables according to rules designed both to protect the data and to make the database more flexible by eliminating redundancy and inconsistent dependency.

DEFINITION:-

NORMALIZATION IS THE PROCESS OF REMOVING OR MINIMIZING REDUNDANCY FROM A TABLE.

Name	Age	Class	Division
Sam	13	7 th	A
Sam	13	7 th	A

IT CAN AFFECT:

INSERTION

UPDATION

DELETION



NORMALIZATION

**REMOVAL OR
REDUCTION
OF DUPLICATE
DATA**

Example

- If Row 2 is updated, row 4 is also updated as they have the same values.
- This will make the data of row 4 incorrect. Remember that we are only updating row 2 and not row 4.

Table name: emp

Employee_name	Department_name	Salary
Kevin	Research	15,000
Michael	Production	18,000
Samuel	Development	24,000
Michael	Production	18,000

SAME

SAME

**UPDATE SALARY VALUE OF
ROW 2 TO 20,000**

UPDATE QUERY: update emp set Salary = 20000 where Employee_name = 'Michael' ;

Addressing Anomalies through Normalization

Table Name: emp

e_id	name	age	dept_id	dept_name	dept_head	dept_phone
1	abc	34	7	Design	pqr	123
2	mno	27	7	Design	pqr	123
3	xyz	26	10	Production	tuv	987
4	slr	40	10	Production	tuv	987
5	smg	35	3	R & D	ksi	555
6	dmr	29	3	R & D	ksi	555

**HOW NORMALIZATION
CAN SOLVE THESE
ANOMALIES?**

Table Name: emp

e_id	name	age	dept_id
1	abc	34	7
2	mno	27	7
3	xyz	26	10
4	slr	40	10
5	smg	35	3

F.K.

Table Name: dept

dept_id	dept_name	dept_head	dept_phone
7	Design	pqr	123
10	Production	tuv	987
3	R & D	ksi	555

Addressing Anomalies through Normalization

Table Name: emp

e_id	name	age	dept_id
1	abc	34	7
2	mno	27	7
3	xyz	26	10
4	slr	40	10

F.K.

Table Name: dept

dept_id	dept_name	dept_head	dept_phone
7	Design	pqr	123
10	Production	tuv	987
3	R & D	ksi	555
12	Marketing	trp	789

Add New Department without adding new employees.
Is insert anomaly addressed?

Addressing Anomalies through Normalization

Table Name: emp

F.K.

e_id	name	age	dept_id
1	abc	34	7
2	mno	27	7
3	xyz	26	10
4	slr	40	10
5	smg	35	3

Table Name: dept

dept_id	dept_name	dept_head	dept_phone
7	Design	pqr	123
10	Production	tuv	987
3	R & D	ksi	555
12	Marketing	trp	789

Deleting Employees: If Employee smg leaves the company, will the department where smg is assigned will also be deleted?

Addressing Anomalies through Normalization

Table Name: emp

F.K.

e_id	name	age	dept_id
1	abc	34	7
2	mno	27	7
3	xyz	26	10
4	slr	40	10
5	smg	35	3

Table Name: dept

dept_id	dept_name	dept_head	dept_phone
7	Design	pqr	123
10	Production	tuv	987
3	R & D	ksi	555
12	Marketing	trp	789

Update anomaly: If department head pqr is changed to mpr, only records of dept_id 7 will be affected.

The importance of normalization

- If a table has data redundancy and is not properly normalized, then it will be difficult to handle and update the database, without facing data loss.
- It will also eat up extra memory space and Insertion, Update, and Deletion Anomalies are very frequent if the database is not normalized.

Levels of Normalization

- First Normal Form (1NF)
- Second Normal Form (2NF)
- Third Normal Form (3NF)
- Boyce-Codd Normal Form (BCNF)
- Fourth Normal Form (4NF)
- Fifth Normal Form (5NF)

First Normal Form

What is First Normal Form?

- A relation is in first normal form if every attribute in that relation is single-valued attribute.
- ✓ No Composite attribute
- ✓ No Multi-valued Attribute

First Normal Form

A)
MUST NOT CONTAIN
COMPOSITE ATTRIBUTE ?

Composite Attribute:

It is an attribute which can be further divided into parts

B)
MUST NOT CONTAIN
MULTI VALUED ATTRIBUTE ?

Multi valued Attribute:

It is an attribute which can have multiple values for single entity

NO



ID	Name	Mobile Number
1	ABC DEF GHI	12345678, 876554321
2	JKL MNO PQR	11223344
3	RST UVW XYZ	21436587, 78563412



making IT simple

ID	First Name	Middle Name	Last Name	Mob Number	Alternate Mob No.
1	ABC	DEF	GHI	12345678	876554321
2	JKL	MNO	PQR	11223344	NULL
3	RST	UVW	XYZ	21436587	78563412

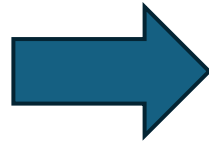
IS IN FIRST NORMAL FORM

First Normal Form

- Is this an optimal solution?

MULTIVALUED ATTRIBUTE

ID	Name	Mobile Number
1	ABC	12345678, 876554321
2	JKL	11223344, 12442135
3	RST	21436587, 78563412
4	SMR	36925814, 74854875, 14251542
5	ASM	24867159



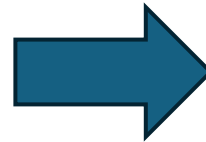
ID	Name	Mob Number	Alternate Mob No.	One More Mob No.
1	ABC	12345678	876554321	NULL
2	JKL	11223344	12442135	NULL
3	RST	21436587	78563412	NULL
4	SMR	36925814	74854875	14251542
5	ASM	24867159	NULL	NULL

First Normal Form

- Is this an optimal solution compared to the first solution?

MULTIVALUED ATTRIBUTE

ID	Name	Mobile Number
1	ABC	12345678, 876554321
2	JKL	11223344, 12442135
3	RST	21436587, 78563412
4	SMR	36925814, 74854875, 14251542
5	ASM	24867159



P.K.

ID	Name
1	ABC
2	JKL
3	RST
4	SMR
5	ASM

F.K.

ID	Mob Number	Employee ID
1	12345678	1
2	876554321	1
3	11223344	2
4	12442135	2
5	21436587	3
6	78563412	3
7	36925814	4
8	74854875	4
9	14251542	4
10	24867159	5

First Normal Form

Note:

- *A database design is considered as a bad design if it is not even in the First Normal Form (1NF).*

First Normal Form

What is the significance of 1NF in database design?

- *The implementation of 1NF is essential for the preservation of data integrity and productivity.*
- *It eliminates the need for redundant data and ensures that the data is structured in a manner that facilitates simple queries and operations.*

Second Normal Form

What is 2nd Normal Form?

- The second Normal Form (2NF) is based on fully functional dependency.
- The second Normal Form applies to relations with composite keys, that is, relations with a primary key composed of two or more attributes.
- A relation with a single-attribute primary key is automatically in at least 2NF.
- A relation that is not in 2NF may suffer from the update anomalies.

What is 2nd Normal Form?

FIRST NORMAL FORM

CONDITION 1:
NO MULTIVALUED ATTRIBUTE

CONDITION 2:
NO COMPOSITE ATTRIBUTE

SECOND NORMAL FORM

CONDITION 1:
**TABLE SHOULD BE IN
FIRST NORMAL FORM**

CONDITION 2:
**TABLE MUST NOT CONTAIN
PARTIAL DEPENDENCY.**

making IT simple



What is a Functional Dependency?

- A functional dependency is a relationship between two sets of attributes in a database, where one set (the determinant) determines the values of the other set (the dependent).
- For example, in a database of employees, the employee ID number (determinant) would determine the employee's name, address, and other personal information (dependent).
- This means that, given an employee ID number, we can determine the corresponding employee's name and other personal information, but not vice versa.

What is a Functional Dependency?

- Functional dependencies can also be represented using mathematical notation.

Employee ID $\rightarrow$ Employee Name, Address, etc.

- It's important to note that functional dependencies only apply to the individual tuples in the table, and not to the table as a whole.

Functional Dependency

A	B
1	ABC
2	DEF
3	GHI
4	JKL

Table T

DETERMINANT

DEPENDANT



IS A FUNCTIONAL DEPENDENCY

VALUE
OF A

DETERMINE

VALUE
OF B

2

DEF

A DETERMINES B

B IS DETERMINED BY A

Functional Dependency

STUDENT TABLE

Roll_No	Student_Name	Dept_Name	Dept_Building
2	abc	CS	A4
3	pqr	IT	A3
4	xyz	CS	A4
5	xyz	IT	A3
6	mno	EC	B2
7	jkl	ME	B2

FUNCTIONAL DEPENDENCY

ROLL_NO $\longrightarrow$ { STUDENT_NAME, DEPT_NAME
DEPT_BUILDING }

VALID FUNCTIONAL DEPENDENCY

making it simple **3** $\longrightarrow$ { PQR, IT, A3 }



Determinant	Dependant
1	a
2	b
VALID F.D.	

Play (k)



Functional Dependency

Roll_No	Student_Name	Dept_Name	Dept_Building
2	abc	CS	A4
3	pqr	IT	A3
4	xyz	CS	A4
5	xyz	IT	A3
6	mno	EC	B2
7	jkl	ME	B2

FUNCTIONAL DEPENDENCY

DEPT_NAME → DEPT_BUILDING

VALID FUNCTIONAL DEPENDENCY

making IT CS → A4

Determinant	Dependant
1	a
1	a

CS → A4

Functional Dependency

Roll_No	Student_Name	Dept_Name	Dept_Building
2	abc	CS	A4
3	pqr	IT	A3
4	xyz	CS	A4
5	xyz	IT	A3
6	mno	EC	B2
7	jkl	ME	B2

Determinant	Dependant
1	a
2	a
VALID F.D.	

FUNCTIONAL DEPENDENCY

DEPT_NAME $\longrightarrow$ DEPT_BUILDING

VALID FUNCTIONAL DEPENDENCY

EC



B2

ME



B2

ROLL_NO $\longrightarrow$ STUDENT_NAME

Functional Dependency

Roll_No	Student_Name	Dept_Name	Dept_Building
2	abc	CS	A4
3	pqr	IT	A3
4	xyz	CS	A4
5	xyz	IT	A3
6	mno	EC	B2
7	jkl	ME	B2

Determinant	Dependant
1	a
1	b
INVALID F.D.	

FUNCTIONAL DEPENDENCY

STUDENT_NAME $\longrightarrow$ DEPT_NAME

INVALID FUNCTIONAL DEPENDENCY

XYZ



CS

XYZ



IT

Functional Dependency

- 4 cases of functional dependency:

Determinant	Dependant
1	a
2	b
Valid F.D.	

Determinant	Dependant
1	a
1	a
Valid F.D.	

Determinant	Dependant
1	a
2	a
Valid F.D.	

Determinant	Dependant
1	a
1	b
Invalid F.D.	

Important points on 1st and 2nd Normal Form

- 1NF is centered on the removal of repeating groups, and 2NF is associated with the issue of partial dependencies.
- With the aid of these normal forms, the databases yield high efficiency and can withstand peculiarities viewed as anomalies, thereby helping in the proper organization of databases.

Partial Dependency

Table R

A	B	C	D
SM	LQ	GT	HY
.	.	.	.
.	.	.	.
.	.	.	.

CANDIDATE KEY :- {A, B}

B IS NOT A CANDIDATE KEY

PRIME ATTRIBUTE

A, B

NON PRIME ATTRIBUTE

C, D

DETERMINANT $\{A, B\}^+ = \{D\}$ DEPENDANT

making IT simple

DETERMINANT $B^+ = \{C\}$ DEPENDANT

PART OF
CANDIDATE
KEY

NON PRIME
ATTRIBUTE

PARTIAL DEPENDENCY

Partial Dependency

CONDITION 2: THERE MUST NOT BE ANY PARTIAL DEPENDENCY

Student_ID	Course_ID	Course_Fee
1	IOT	750
2	IOT	750
3	IOT	750
1	AI	880
2	AI	880
3	AI	880

(Student_ID, Course_ID)
IS A CANDIDATE KEY

IF A NON-PRIME ATTRIBUTE IS DETERMINED
BY A PART OF CANDIDATE KEY, THAT FUNCTIONAL
DEPENDENCY IS PARTIAL DEPENDENCY

(Student_ID, Course_ID) → Course_Fee
COMPLETE CANDIDATE KEY NON-PRIME
ATTRIBUTE

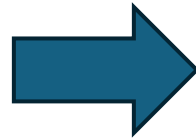
PARTIAL DEPENDENCY
Course_ID → Course_Fee
PART OF CANDIDATE KEY NON-PRIME
ATTRIBUTE

Prime Attributes	Non-Prime Attributes
Student_ID	Course_Fee
Course_ID	



Second Normal Form

Student_ID	Course_ID	Course_Fee
1	IOT	750
2	IOT	750
3	IOT	750
1	AI	880
2	AI	880
3	AI	880



Student_ID	Course_ID
1	IOT
2	IOT
3	IOT
1	AI
2	AI
3	AI

Course_ID	Course_Fee
IOT	750
AI	880

Partial Dependency

DEFINITION



Partial Dependency in DBMS occurs when a non-prime attribute depends on the proper subset or we can say part of the candidate key.

STEPS :-

- 1) IDENTIFY CANDIDATE KEYS
- 2) DIFFERENTIATE PRIME AND NON-PRIME ATTRIBUTES
- 3) FIND A NON-PRIME ATTRIBUTE THAT IS DETERMINED BY A PART OR PROPER SUBSET OF CANDIDATE KEY

Convert to 2nd NF

- **Business Rule:** In a university database, each course can be taught by multiple professors, and a professor can teach multiple courses. The Course table stores the course code, professor ID, and semester.

<u>Course Code</u>	<u>Professor ID</u>	Semester	<u>Professor Name</u>
CCE101	P001	FirstSem2024	Sir Misa
CCE 101	P002	FirstSem2024	Sir <u>Atiagan</u>
CCE 107	P001	SecondSem2024	Sir Misa

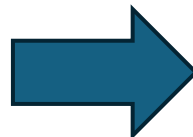
Normalized Table (in 2NF):

Course Table

Course_Code	Semester
CCE101	FirstSem2024
CCE 107	SecondSem2024

Professor Table

Professor_ID	Professor_Name
P001	Sir Misa
P002	Sir Atiagan



Normalized Table (in 2NF):

Course Table

Course_Code	Semester
CCE101	FirstSem2024
CCE 107	SecondSem2024

Professor Table

Professor_ID	Professor_Name
P001	Sir Misa
P002	Sir Atiagan

Teaching Assignment Table (Junction Table)

Course_Code	Professor_ID
CCE101	P001
CCE 101	P002
CCE 107	P001

Why can Professor ID not be a foreign key in the course table?

Why can Professor ID not be a foreign key in the course table?

Explanation:

Course Table

Course_Code	Semester	Professor_ID (FK)
CCE101	FirstSem2024	P001
CCE 101	FirstSem2024	P002
CCE 107	SecondSem2024	P001
CCE101	SecondSem2024	P001 (added)
CCE101	Summer2024	P001 (added)

You would store P001 (Sir Mlsa) multiple times for different semesters in the same course.

Why can Professor ID not be a foreign key in the course table?

- Making Professor_ID a foreign key in the Course table violates the Second Normal Form (2NF) due to *partial dependency*.

Why can Professor ID not be a foreign key in the course table?

- **How it Violates 2NF:**

- In the Course table, if you include Professor_ID as a foreign key, you create a partial dependency:
 - Professor_ID depends only on Course_Code, not on the full composite key (Course_Code, Semester).
 - Therefore, attributes like Professor_Name (if added) would also depend only on Professor_ID (a part of the key), not on the full key.
- This violates the principle that all non-prime attributes should depend on the entire primary key.

Why can Professor ID not be a foreign key in the course table?

➤ **Correct Approach (Junction Table):**

- Instead of storing Professor_ID in the Course table, you separate it into a Junction Table (Teaching Assignment) where Course_Code and Professor_ID form a composite primary key.

➤ This way:

- Professor_ID depends on the entire key in the Teaching Assignment Table (which is a composite of Course_Code and Professor_ID).
- No partial dependency exists, thus preserving 2NF.

The Role of the Junction Table (Bridge)

A. Represents a Many-to-Many Relationship:

- A single course can be taught by multiple professors.
- A single professor can teach multiple courses.
- The **Teaching Assignment** table acts as the intermediary that connects these two entities, allowing for this many-to-many relationship.

The Role of the Junction Table (Bridge)

B. Stores Unique Combinations:

- The table holds unique combinations of Course_Code and Professor_ID.
- This ensures that each professor-course assignment is recorded only once.

The Role of the Junction Table (Bridge)

C. Composite Primary Key:

- The primary key of the table is the combination of Course_Code and Professor_ID.
- No professor can be assigned the same course twice, preventing duplicates.

The Role of the Junction Table (Bridge)

D. Maintains Data Integrity:

- Course_Code is a foreign key that references the **Course** table, ensuring the course exists.
- Professor_ID is a foreign key that references the **Professor** table, ensuring the professor exists.